

2

Data

Supervised learning and unsupervised learning share much of the same notation and assumptions about data. We focus on supervised learning here to set up the notation and discuss what makes machine learning challenging, but the notation and ideas here hold also for unsupervised learning.

Supervised Learning is the task of turning a dataset of input-output pairs into a general rule. This rule should be able to produce the correct output for future inputs, ones not necessarily seen in the original dataset. In this way, supervised learning seeks to *generalize* from the specific examples provided in the dataset to a rule that is applicable for any example.

The inputs are denoted with x , while their outputs are denoted with y . The function we wish to learn or estimate is denoted f . Thus, we would like $f(x) = y$ for any potential input x .

Further information

In the example in the previous chapter, the input is a subimage, and the output is whether the subimage is that of a hippopotamus's face.

Examples

The input might be the image of a hand-written digit, and the output should be the digit itself (one of 10 categories). Or, the input might be the measurements taken during a hospital stay (for example, the patient's blood pressure, age, and weight; and the results of a chest x-ray), and the output should be the duration of time until the next hospital admission for this patient. Or, the input might be the spectrum captured by a telescope looking at a celestial object, and the output should be the class of object (for example, planet, star, or galaxy). Or, the input might be the text of a restaurant review, and the output should be the numeric rating of the restaurant (for instance, on a scale of 1-to-10).

For each of these scenarios, we assume that we have a dataset of examples of corresponding input-output pairs. For the digit-recognition scenario, this might be scanned images from people's handwritten notes, where each digit image has been carefully cropped

out and annotated with the correct digit by a human. For the hospital-readmittance scenario, these examples might be collected from electronic health records, with one example for each hospital stay, connecting it to the time to the next hospital stay. For the celestial-object-classification example, the spectra could be collected from sky surveys, with the associated classes taken from human annotations. The dataset for the restaurant reviews could be constructed from online reviews, pairing the text with the supplied rating for each review.

Data Format

This dataset, the information used for learning, is called the **training set**. It consists of a set of examples. Each example consists of an input and output. The output is also known as the **label**, or **target**. The input has a set of **features** (also called **attributes**). For the digits, these might be the values of each pixel in the image. For the hospital records, these features are the different vital signs or tests measured. For the sky classification, these would be the intensities at each wavelength in the spectrum. For the restaurant reviews, the features could be the counts of the number of times particular words appear in the review text.

The training set has a fixed number of examples, which we denote as m . We let $\vec{x}_i$ and y_i represent the input and output (respectively) of the i th example in the training set. So, $\vec{x}_1$ is the input of the first example and its corresponding output is denoted y_1 . Similarly, the 501st example in the training set consists of $\vec{x}_{501}$ and y_{501} .

We let n denote the number of features in the training set. $\vec{x}_i$ is a vector of each of these features' values of example i . It has elements $x_{i,1}, x_{i,2}, \dots, x_{i,n}$. For example, $x_{3,5}$ is the 5th feature for the 3rd example.

A (new) input, not necessarily part of the training set, is denoted $\vec{x}$. Unfortunately, this means that its j th feature is written x_j , which looks very similar to the full vector of all of the features of the j th training example: $\vec{x}_j$. Frequently, both notations are used in the same writing, even when omitting the arrow above vectors: For example x_2 can refer to either the 2nd feature of a general example or to the vector of all of the features of the 2nd example in the training set. There is not an agreed-upon notion for avoiding this because, for those familiar with machine learning, the intention can generally be inferred from context. Here, we keep the arrow on $\vec{x}_j$ to help differentiate.

These feature vectors and outputs are collected into two matrices X and Y . X is of shape m -by- n : Each row is an example and each column is an input feature. Y is of shape m -by-1: Each row is an

Further information

Determining a good set of features is a critical part of the set-up of a supervised learning problem. It is important to understand how the machine learning methods work to understand what features (input) might work well. Thus, after covering the methods, we will return to this problem in later chapters.

Further information

The arrow on top of $\vec{x}_i$ reminds us that this is a vector. Bold can also be used to indicate a vector. Outside of this text, the arrow (or bold) is usually omitted.

Key point

Be careful not to confuse $\vec{x}_j$ with x_j .

example (in the same order as in X) and there is only one output value. Thus the vector $\vec{x}_1$ (the features of the first example) “lies down” across the first row of X . We can denote this by taking the transpose of $\vec{x}_1$: $\vec{x}_1^\top$.

Schematically, this means that the matrices X and Y look like

$$X = \begin{bmatrix} \text{---} \vec{x}_1^\top \text{---} \\ \text{---} \vec{x}_2^\top \text{---} \\ \vdots \\ \text{---} \vec{x}_m^\top \text{---} \end{bmatrix} \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} \quad (2.1)$$

A concrete example is shown in Table 2.1 and just below.

Terminology

Features and outputs are broadly categorized into one of three types: (a) continuous, (b) categorical, and (c) ordinal. Continuous features or outputs take on numeric values on the real number line. Examples of such features include height, temperature, speed, or mass. Categorical features or outputs take on values from a (generally small) set of possible values. Examples of such features include hair color, whether a creature has wings, and class of road vehicle (for example, truck, car, motorcycle). Ordinal features or outputs also take on values from a discrete set of possibilities. However, these possibilities are ordered. An example is a rating on the Likert scale (“strongly disagree,” “disagree,” “neither agree nor disagree,” “agree,” and “strongly agree”).

The supervised learning problem itself is often categorized by the type of the output. If the output is continuous, the problem is known as **regression** and the function f is called a **regressor**. If the output is categorical, the problem is known as **classification** and the function f is called a **classifier**. A special sub-category of classification is **binary classification**, when the output can take only one of two values (for example, “yes” or “no”). If the output is an ordinal value, it is known as **ordinal regression**, but this is not a topic covered here.

Running Example

To make the above notation concrete, consider the problem of determining the land speed of a hippopotamus from a distance.¹ You have, at great personal risk, gathered a dataset of measured values from different hippopotamuses. In particular, for each of the 10 hippopotamuses, you measured three things: its girth (measured in meters)², its ear wiggle rate (measured in Hertz), and its maximum land running speed (measured in kilometers per hour). It is the last

¹ An important estimation task if you are on the African savanna and need to know how much of a head-start you might need should a hippopotamus charge.

² Hippopotamuses do not like to talk about their weight, so we use girth.

girth	ear wiggle rate	speed
3.49	0.41	12.01
2.39	0.34	8.49
3.28	0.14	17.81
3.10	0.31	25.67
3.01	0.27	22.89
3.11	0.53	26.09
2.57	0.53	18.53
3.07	0.38	24.47
2.54	0.36	15.76
3.64	0.05	8.98

Table 2.1: Running example data. No hippopotamuses were harmed in the creation of these data.

that we would like to predict from the former two. The data are as in Table 2.1.

Each row corresponds to a hippopotamus (an example). The left two columns correspond to the input features (girth and ear wiggle rate) the right column corresponds to the associated output (speed). When split into X and Y it looks like

$$\begin{array}{lcl}
 \begin{array}{l} \text{features of fourth} \\ \text{hippopotamus} \end{array} & \begin{array}{c} X = \begin{bmatrix} 3.49 & 0.41 \\ 2.39 & 0.34 \\ 3.28 & 0.14 \\ 3.10 & 0.31 \\ 3.01 & 0.27 \\ 3.11 & 0.53 \\ 2.57 & 0.53 \\ 3.07 & 0.38 \\ 2.54 & 0.36 \\ 3.64 & 0.05 \end{bmatrix} \\ \text{girth of ninth} \\ \text{hippopotamus} \end{array} & \begin{array}{c} \begin{array}{c} \text{second hippopotamus} \\ \text{target for fourth} \\ \text{hippopotamus} \end{array} \\ Y = \begin{bmatrix} 12.01 \\ 8.49 \\ 17.81 \\ 25.67 \\ 22.89 \\ 26.09 \\ 18.53 \\ 24.47 \\ 15.76 \\ 8.98 \end{bmatrix} \end{array} \\
 & & (2.2)
 \end{array}$$

where a few of the elements have been labeled to help connect to the notation above.

These are the hippopotamuses we collected information about and from which we will estimate a relationship between the input feature pair of girth and ear wiggle rate and the output target of speed. Once this is done, we will (no doubt) come across a new hippopotamus. As a “generic” example, its features would be denoted as the vector $\vec{x}$. If we (perhaps later, at personal peril) find its true running speed, as a “generic” future example, this would be denoted as y . So, a general

example (not necessarily part of the training set) might look like

$$\begin{array}{c} \text{ear wiggle rate} \end{array} \vec{x} = \begin{bmatrix} 2.97 \\ 1.13 \end{bmatrix} \quad x_2 \quad y = 24.82 \text{ speed} \quad (2.3)$$

Note that $\vec{x}$ is a column vector, $\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$. Its elements are x_1 and x_2 . In the training set, $\vec{x}_4$ (for example) is also a column vector. But, it is “lying down” in the matrix X and therefore (for example above), as a row of X it is $\vec{x}_4^T$. The girth of the fourth hippopotamus in the training set is $x_{4,1}$. The girth of this general hippopotamus immediately above is x_1 , which is different than the entire vector of features of the first training hippopotamus which is $\vec{x}_1$.

Data Assumptions

We assume rows in our training data are not in any special order. For example, we assume that the ten hippopotamuses in the above example could have been reordered (shuffling the rows of X and Y each the same way) without losing any information. This extends to the future examples as well. That is, we assume that whether a particular hippopotamus is included in the training set or is left out (and might appear as a future example upon which our function f will be employed) is completely at random.

These assumptions are captured mathematically when we say that the $\vec{x}$ - y pairs are drawn at random from a distribution $\Pr(\vec{x}, y)$ and that each example in the training set is drawn *independently* (whether a particular hippopotamus appears in the training set does not influence whether a different hippopotamus will appear) and *identically distributed* (all hippopotamuses’ features are drawn from the same distribution, $\Pr(\vec{x}, y)$). Future examples are also drawn *independently and identically distributed* from this same distribution.

This assumption is often false. However, this does not invalidate the use of machine learning. As with any model, it is impossible for it to capture all aspects of the true situation. Some assumptions must be made to simplify the problem. We are explicit with the assumptions so that we can judge to what degree they hold (or do not hold) in the scenario at hand. This gives us confidence in deciding when the results are valid, and it allows us to use the tool properly in the right circumstance.

We may have collected our data about the land speed of hippopotamuses by visiting only a few watering holes. Our data were gathered from hippopotamuses that were, most likely, genetically related. Further, they may have been listed in the order in which they were observed, and thus blocks of consecutive rows in our data may be from the same watering hole. This certainly means that the

Further information

For almost every general statement about machine learning (like “we assume that...”), there are exceptions. Someone has questioned the assumption and developed a branch of ML that does not make that assumption. But, we are focusing on the most common formulations.

Key point

This is called the *i.i.d.* assumption.

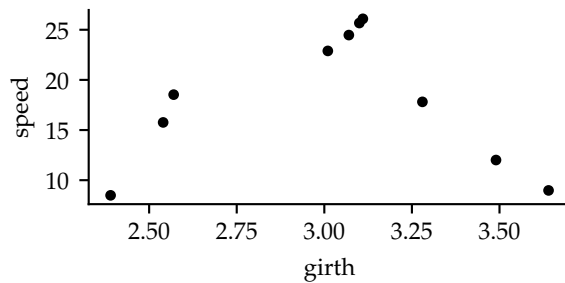


Figure 2.2: Data from Table 2.1, omitting second feature (ear wiggle rate). Each point represents one example (hippopotamus). Never before has a hippopotamus been quite so small.

data are not *i.i.d.*: a mother is more likely to be in the sample if her offspring are also in the sample (not independent) and blocks of consecutive data come from the same sub-distribution (not identically distributed).

If we randomly permute the rows of our data, we have restored the identically distributed property within the training data. But, whether or not our training data are independently drawn from the same distribution as the future data we will see (the key part of the *i.i.d.* assumption) depends on how well the populations of these few watering holes represent the general population of hippopotamuses we will see in the future. If we will only see hippopotamuses from these same watering holes again or if these watering holes are representative of all watering holes, our assumption is valid. If we are likely to employ the learned function f on hippopotamuses from other watering holes where the relationships among girth, ear wiggle rate, and speed are significantly different from those of hippopotamuses from the watering holes represented in the training data, then we should have little faith in the output of f generalizing to these new hippopotamuses.

Throughout this text, we assume *i.i.d.* data. Confirming this assumption is important, but very particular to the application domain and therefore outside the scope of what is covered here.

Why is this hard?

Consider a slightly simpler example where we only try to predict the speed from the girth of the hippopotamus. Thus, we use the same data as in Table 2.1, but remove the middle column. The data can be plotted as in Figure 2.2.

Our goal is to create a function f that can predict speed from girth. In subsequent chapters we will discuss how to do this. For

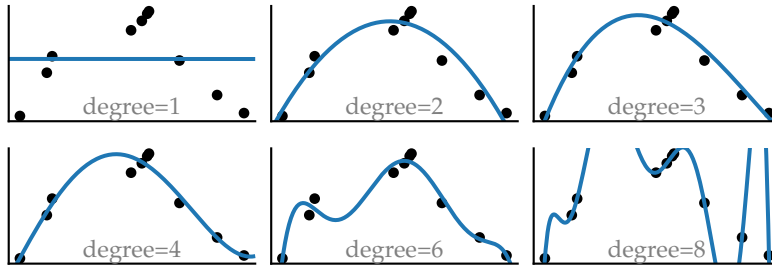


Figure 2.3: Fit of data from Figure 2.2, with different degree polynomials. f is shown in blue.

the moment, we will just consider the result. Figure 2.3 shows the result of estimating f six different ways: Each assumes f is a different degree of polynomial. That is, the first plot shows the best-fitting linear function and the last plot shows the best-fitting eighth degree polynomial.

The second, third, and fourth degree polynomials seem (visually) to be good approximations of the relationship between girth and speed. However, the other polynomials (of greater or less degree) do not fit the data well. In the case of the first-degree polynomial (line), the function is not flexible enough. It cannot capture the important relationship in the data: that the largest and smallest hippopotamuses are slower and the middling-girth hippopotamuses are the fastest. We often refer to this as **underfitting**. In the case of high-degree polynomials, the function is too flexible. It fits not only the important relationship between girth and speed, but it also fits what we would call “noise.” We often refer to this as **overfitting**.

This is a general pattern that occurs frequently in machine learning. We have a choice (in this case what degree of polynomial to use). For some settings this choice, we overfit. For other settings, we underfit. The difficulty is in picking this hyper-parameter properly.

In this example, it does not appear difficult to pick the polynomial degree: We can just look at the plots and see when f is “too wiggly” or “not wiggly enough.” But, in general we cannot just plot the data and pick by eye. Usually, there are many many features (tens, hundreds, or thousands). The plots above are two-dimensional because there are two quantities of interest: girth and speed. If we increase the number of features (things we measure about hippopotamuses), we increase the dimensionality of this plot. Past about three dimensions, this becomes impossible for humans to visualize. Further, often the selection must be performed by the computer without aid from a human (for instance, because of the speed at which it must be completed).

Further information

The choice of polynomial degree is formally called a hyper-parameter, which we define and discuss later.

Further information

Throughout this text, we will show small examples with only a few features. While these are helpful for learning, they can also lead to invalid intuitions. Real problems are in high-dimensional spaces that do not adhere to our intuitions developed in three-dimensional space. Further, such examples often lead to reasoning “but, I could just do X” when “X” is not feasible by a computer that needs hard-and-fast rules that cannot rely on a visual system. Be cautious.

Consider the “degree=6” plot in Figure 2.3. With your human-based knowledge about the quantities plotted (girth and speed), you would probably say that this has overfit: the dip in the middle seems unlikely. That the three points in the middle have an upward trend is due to noise in the data and the function should not have a steep upward slope there, causing a dip just to the left of it. Yet, if the features were different (but the data the same), we might have a different conclusion. Consider if the x -coordinate were the position along a street and the y -coordinate were the amount of pedestrian traffic. The upward trend in the center might be justified by a series of shops whose customers all arrive and depart from the right. This is, admittedly, a contrived scenario. Yet it illustrates that the computer knows no semantic meaning of the features or the label. It is asked to produce f without external knowledge about the variables involved.

The terms underfitting and overfitting are used by humans to explain general concepts. There is no hard-and-fast definition of when a machine learning method is underfitting or overfitting. The “noise” in the data is a similarly nebulous concept. The points in Figure 2.2 do not lie exactly on a nice curve. Do their speeds deviate from an “ideal function” due to measurement inaccuracies, due to other factors not measured (for example, cardio-vascular capacity), or due to inherent randomness in the world? Which explanation is correct is unknowable from the data and largely a philosophical question. We treat all of these as noise and model it as inherent randomness of the world.

Yet, perhaps the deviations are explained by the other measured features (ear wiggle rate, in this case), but are just not apparent in this visualization of the data. In this case, the information is available to the machine learning algorithm and is not considered noise. If this relationship (among speed, girth, and ear wiggle rate) is complicated enough, we might not be able to tease it out from the limited data we have. In that case, we might as well treat some of the variation as noise, or we risk overfitting. As more features are measured, the potential complexity of interactions grows. Thus, often we wish to pick a model that is less complex than reality, in order to prevent overfitting on limited amounts of data.

Common Approach

A common approach to develop a machine learning algorithm consists of three steps:

1. Select a hypothesis space.
2. Select a definition of error.
3. Develop an algorithm that selects the member of the hypothe-

sis space that minimizes the error on the training set.

The **hypothesis space** is the set of functions we will consider for f . In each of the six examples in Figure 2.3 there is a different hypothesis space. In the first plot, the hypothesis space is all possible linear functions (a large set!). In the second plot, the hypothesis space is all possible quadratic functions (an even larger set!). Other hypothesis spaces are possible: the set of all computer programs of if-then-else chains, for example.

The error is also often called the **loss** or **risk**. It measures how badly a prediction (from f) matches a true value. For all of the examples in Figure 2.3, we used squared error: the square of the difference between the predicted value and the true value.

Armed with a hypothesis space and an error function, we develop a method to minimize this error on the training data. This is the machine learning algorithm. It takes in a dataset and outputs a function f (often implemented itself as an algorithm). We may slightly modify this procedure to try to combat under- and overfitting, but this approach encompasses most of the methods in this text. For the examples in Figure 2.3, the function f was found using the methods described in Chapter 4.

Testing

Consider the problem of running HipBook, a social network website for hippopotamuses. You have collected data from some of your users (number of friends, marital status, number of hours they sleep per day, and whether they enjoy eating alfalfa). You would like to build a function to predict whether a user likes eating alfalfa based on the other attributes so that you can present advertisements for alfalfa to those who enjoy it. So, you plug your collected data as the training set into a machine learning algorithm and get out a prediction function, f . How do you know if f will work well?

You could check to see how well f performs on the training set. But, consider that in Figure 2.3, the eighth-degree polynomial fits the training set very well (f 's predicted value is exactly correct for each of the hippopotamuses in the training set). Yet, we would not expect this function to *generalize* well to future, as of yet unseen, examples.

The true measure is how well f will perform in the future. Therefore, we could use f in HipBook in the future to present advertisements and compare its performance to another method (or to past attempts). Yet, often we do not have the time to wait for future results. Instead, we use a **testing set**. This is a dataset of the same form as the training set: same features for $\vec{x}$ and same output for y , but for a different collection of examples. It may or may not have the same

Key point

Performance on the training set is not necessarily indicative of performance on future examples.

number of examples as the training set. We can check the accuracy of our predictions on this testing set as a proxy for future examples.

For this method to be valid, it is important that the testing set *only* be used once all of the machine learning is complete. If the testing set is used to steer or guide the selection of machine learning method or the optimization of the function f itself, then it has, de facto, become part of the training set: part of the information used to pick f .

Key point

Do *not* use the testing set for anything other than testing the *final* result of machine learning. See Chapter 6 for how to select among different hypothesis spaces based only on the training set.

Causality

Finally, nothing in this text addresses finding causation. There are branches of machine learning that address causality. They are not covered here.

The speed of a hippopotamus may be predictable from its girth. However, this does not mean the girth causes the speed. It might be (certain sized hippopotamuses are better mechanically at running). However, it might be the reverse (speedy hippopotamuses have different metabolisms that result in medium-valued girths). Or, it might be that neither causes each other and they are both caused by a third factor (longer-legged hippopotamuses can run faster and their longer legs allow them to reach food that gives them a middling waistline). Without interventional experiments or further statistical assumptions (and data) about hippopotamuses, we cannot distinguish among these possibilities.

Key point

Make *no* conclusions about causes or effects from any of the results of the algorithms in this text.